

ourTailTales

MVP Product + Technical Build Specification

A simple funnel that turns a pet photo album into a printed life story

ourTailTales.com

Core product promise

Drop in their album. ourTailTales organizes the life timeline, proposes chapters, builds the book, previews it, and sends the paid order directly to Lulu for printing and fulfillment.

Primary implementation target: Next.js + TypeScript + Supabase + Stripe + one AI provider + Lulu Print API

This document is intentionally opinionated. Build the smallest sellable funnel first; do not turn V1 into a general-purpose photo editor.

1. Company vision

ourTailTales turns the scattered camera roll of a pet's life into a physical, chronological keepsake. The software is not the end product; the physical book is. The software exists to remove the work between "I have hundreds of photos" and "I have a finished memorial book."

Positioning

- **Customer problem:** Owners have years of pet photos, but creating a coherent memorial book manually is emotionally and practically difficult.
- **ourTailTales solution:** Use photo metadata and lightweight AI to reconstruct a timeline, propose chapters, choose representative photos, write short chapter introductions, and lay out a print-ready book.
- **Business model:** One-time physical-book purchase. No subscription is required for the MVP.
- **Emotional standard:** AI organizes and summarizes evidence from the album; it must not invent specific memories as fact.

MVP success definition

A successful V1 is not "feature complete."

A stranger can land on ourTailTales, drop in hundreds of photos, get a credible book without creating an account, preview it, pay, and trigger a valid Lulu print job without manual intervention.

Explicit non-goals

- No user accounts, profiles, dashboards, saved libraries, social features, subscriptions, or collaboration.
- No permanent storage of the uploaded album.
- No AI caption generation on every page.
- No free-form Canva-style editor. Users may swap/reorder photos and edit text, but layouts come from ourTailTales templates.
- No multiple book formats in V1. One physical product only.

2. Product rules

Rule	MVP decision
Brand	ourTailTales
Physical product	8.5 x 8.5 in square hardcover casewrap
Interior	Premium full color on 80# coated white paper
Cover	Casewrap; default matte finish unless physical sample proves gloss is better
Base price	\$49.99
Base story length	20 story pages = 2 chapters x 10 pages
Lulu minimum handling	Base print file contains 24 interior pages: 20 story pages + 4 fixed pages
Upsell	Each additional chapter adds 10 story pages and \$10
Typical order	4 chapters / 40 story pages / 44 Lulu interior pages / \$69.99 before shipping/tax
Photos per chapter	Target 20-30 selected photos
AI output	One chapter title + short opening blurb per chapter
Preview	Interactive browser preview + downloadable first 5 pages with ourTailTales watermark after email capture

Pricing function

```
const BASE_CHAPTERS = 2;
const BASE_PRICE = 49.99;
const STORY_PAGES_PER_CHAPTER = 10;
const FIXED_INTERIOR_PAGES = 4;

price = BASE_PRICE + Math.max(0, chapterCount - BASE_CHAPTERS) * 10;
storyPages = chapterCount * 10;
luluInteriorPages = storyPages + FIXED_INTERIOR_PAGES;
```

Important Lulu constraint

Lulu’s 8.5 x 8.5 hardcover casewrap supports a 24-page minimum. ourTailTales therefore markets “20 pages included,” while the actual print file contains 4 additional fixed pages so the base book is valid for Lulu.

3. End-to-end customer funnel

Keep the experience on the landing page as long as possible. The landing page should progressively transform from marketing hero -> processing -> book configuration -> story generation -> editor/preview. The only separate customer-facing route required for the MVP is checkout.

1. Land on ourTailTales. Hero headline, one-sentence explanation, example physical-book mockup, and a large album drag/drop zone are above the fold.
2. Drag/drop or select the entire pet album. Do not require signup first.
3. Process locally in the hero: read metadata, generate small thumbnails, score quality, detect exact/near duplicates, sort chronologically, and cluster by date/location.
4. Ask only what the album cannot reliably supply: pet name; optional birth/death years; optional dedication. Show detected photo count and date range.
5. Show the book-size selector. Default around 4 chapters when enough photos exist. Slider updates chapter count, story pages, approximate photo count, and price in real time.
6. Customer confirms size. ourTailTales deterministically proposes chapter boundaries and preselects photos for each chapter.
7. Customer clicks “Create My Story.” Send only chapter summaries + a few small representative thumbnails per chapter to the AI endpoint. AI returns chapter titles and short blurbs.
8. Build the complete book on the frontend from pre-made templates. User sees a realistic page-turn/thumbnail preview.
9. Allow lightweight corrections: rename chapter, edit blurb, swap a photo, reorder photos inside a chapter, choose a different cover photo, edit dedication.
10. Offer “Download 5-page sample.” Ask for email, save lead, then generate/download a watermarked sample PDF locally.
11. Click “Order My Book.” Generate final Lulu-ready interior PDF and cover PDF in the browser, then upload only those two PDFs to private Supabase Storage.
12. Navigate to /checkout. Collect shipping address, phone, email, shipping method, and payment.
13. Server validates/quotes the order against Lulu, charges through Stripe, and after a successful Stripe webhook immediately submits the print job to Lulu.
14. Show confirmation page with order number. Lulu prints and ships directly to the customer.

Progress-loss behavior

During creation, originals exist only in the current browser tab. Attach a beforeunload handler once processing starts. Browsers do not allow reliable custom close-tab text; show a standard browser warning plus an in-app banner: “Keep this tab open - closing it will lose your current book.”

4. Landing-page state machine

State	Primary UI	Primary action
idle	Hero + example + drag/drop zone	Select album
processing	Progress meter: reading metadata /	Wait; cancel allowed

	thumbnails / sorting	
album_ready	Pet name + detected years/locations + quality summary	Continue
configure	Chapter slider + price + pages + photo estimate	Confirm book size
organizing	Timeline/chapter cards populate	Create My Story
ai_generating	Generate titles/blurbs; clear progress per chapter	Wait/retry failed chapter
editing	Book preview + chapter/photo controls	Preview / sample / order
exporting	Generating final PDFs locally	Continue to checkout

Suggested hero copy

ourTailTales

Turn their camera roll into the story of their life.

Drop in your pet’s photo album. We’ll organize the years, build the chapters, and create a hardcover book you can keep forever.

5. Local photo-processing pipeline

The upload size is variable: a customer might select 50, 100, 800, 2,000, or more photos. The browser must never decode the entire album at once. Keep File references lightweight and process files incrementally in a Web Worker or bounded-concurrency queue so memory use is controlled regardless of album size.

Recommended client libraries

Need	Recommended tool	Why
EXIF / GPS / capture time	exifr	Fast browser-friendly EXIF parsing; supports GPS and dates
Image decode/resize	createImageBitmap + Canvas/OffscreenCanvas	Native; avoids large base64 strings
State	Zustand or React reducer	Simple session-only state
PDF generation	pdf-lib	Runs in browser; precise page sizing and image placement
Hashing	Web Crypto + small dHash helper	Exact duplicate hash + lightweight near-duplicate score

For each uploaded photo

- 1. Read metadata only: DateTimeOriginal / CreateDate, latitude, longitude, orientation, dimensions.

2. If capture date is missing, fall back to file lastModified and mark source as “fallback.” Never pretend it is EXIF.
3. Generate a 384-512 px thumbnail Blob. Never convert all originals to data URLs.
4. Compute exact hash. Optionally compute dHash for near-duplicate grouping.
5. Compute a cheap quality score: resolution, exposure clipping, and edge/sharpness estimate. Mark weak images; do not silently delete them.
6. Store only metadata + thumbnail URL/blob + original File reference in client state.

```
type PhotoAsset = {
  id: string;
  file: File; // current-tab only
  thumbnail: Blob;
  capturedAt: Date | null;
  dateSource: 'exif' | 'lastModified' | 'unknown';
  lat?: number;
  lng?: number;
  width: number;
  height: number;
  orientation: 'portrait' | 'landscape' | 'square';
  qualityScore: number;
  duplicateGroupId?: string;
  selected: boolean;
};
```

Memory rule

Never keep hundreds of decoded ImageBitmaps or base64 strings in state. Decode -> thumbnail/measure -> release. The original File object is a browser handle to the selected file and does not mean the whole image is permanently decoded into RAM.

6. Deterministic chapter creation

AI should not decide the raw chronology. The browser should create the timeline deterministically first; AI only names and summarizes the resulting chapters.

Boundary algorithm

- Sort usable photos by capturedAt.
- For every adjacent pair, calculate a break score from time gap + GPS distance change.
- Select the strongest chapterCount - 1 breakpoints while enforcing reasonable minimum photos per chapter.
- If GPS is sparse, rely mostly on time gaps. If capture dates are sparse, use chronological file order as a weak fallback and tell the user.
- If no strong natural breaks exist, split the timeline into evenly populated chronological segments.

```
breakScore =
  normalize(logDaysBetween(a, b)) * 0.65 +
  normalize(kmBetween(a, b)) * 0.35;

// Pick top N breakpoints, then enforce min photos/chapter.
```

Representative photo selection

- Target 20-30 photos per chapter for 10 pages.
- The number of uploaded photos does not determine price. The customer's chosen chapter count determines the book's page and photo budget. For each 10-page chapter, select approximately 20-30 representative photos from the full album; photos beyond that budget remain available as swap candidates but are not

placed in the book. If the album has fewer usable photos than the target, use the available photos without duplicating them and adjust layouts toward larger images.

- Divide each chapter timeline into small temporal bins and pick the highest-quality image(s) from each bin so one afternoon does not dominate a multi-year chapter.
- Penalize duplicate/near-duplicate images and repeated GPS/time clusters.
- Prefer orientation diversity because it improves layout variety.
- User can always replace the algorithm’s choice with another photo from that chapter.

7. AI story generation

Use one server-side Next.js route so the provider API key never reaches the browser. Keep the provider/model configurable via environment variables.

What is sent per chapter

- Pet name and optional lifespan.
- Chapter start/end dates.
- Representative location clusters (coordinates plus reverse-geocoded label when available).
- Counts: photos, recurring locations, seasonal/time patterns.
- 3-5 compressed representative thumbnails, not every photo.
- Any user-entered dedication/context that is explicitly allowed for story generation.

Expected structured response

```
{
  "title": "The Austin Years",
  "dateLabel": "2018-2020",
  "blurb": "...60-110 words...",
  "confidenceNotes": ["Location inferred from GPS metadata"]
}
```

Prompt rule

Do not fabricate memories.

The model may describe visible/metadata-supported patterns and use gentle language such as “These photos trace...” or “This chapter appears to...” It must not assert specific emotions, favorite places, relationships, medical events, or routines unless the user supplied them.

Reverse geocoding

For the MVP, reverse-geocode only representative GPS cluster centroids, not every photo. Put the provider behind one tiny server helper so it can be changed later. If geocoding fails, the book can still use dates and omit place names.

8. Book composition and templates

Each chapter is exactly 10 story pages. The chapter opener counts as one of those 10 pages. This keeps pricing and pagination deterministic.

Page type	Purpose	Typical content
Fixed page 1	Title page	Pet name + lifespan + small hero image

Fixed page 2	Dedication	Optional owner dedication; tasteful blank alternative
Chapter opener	1 per chapter	Chapter title, date range, 60-110 word blurb, 1 hero image
Photo pages	9 per chapter	Pre-made layouts; generally 2-3 images/page
Fixed page 3	Closing page	Large final image + short fixed line such as “Always part of the story.”
Fixed page 4	Imprint/blank	ourTailTales mark + production note or intentional blank

Layout set - V1

- Full-bleed single photo.
- One large photo + two small photos.
- Two-photo vertical split.
- Two-photo horizontal split.
- Three-image editorial grid.
- Four-image grid.
- Chapter opener: text block + hero image.

Layout selection rules

- Choose template based on the orientation mix of the next photos.
- Do not crop faces/subjects aggressively; use object-fit contain/fill rules with user-adjustable focal position only if necessary.
- Avoid using the exact same layout more than twice in a row.
- Text is limited to chapter opener, dedication, cover, and optional metadata line. No AI-generated caption on every page.

9. Editor, preview, and lead capture

The editor must feel like reviewing a finished book, not designing one from scratch.

Allowed controls

- Edit pet name, lifespan, dedication, chapter title, and chapter blurb.
- Swap a selected photo with another photo from the same chapter.
- Drag to reorder photos within a chapter.
- Choose cover photo.
- Regenerate one chapter blurb, not the whole book.
- Optional: choose between 2-3 cover themes later; not required for first build.

5-page sample

1. User clicks “Get my 5-page sample.”
2. Modal asks for email only. Explain that the sample downloads immediately.
3. POST email + minimal funnel metrics to /api/leads.

- 4. Generate a PDF locally from the first 5 interior preview pages at screen-quality resolution.
- 5. Overlay a repeated or diagonal “OURTAILTALES PREVIEW” watermark on every sample page.
- 6. Trigger browser download. Do not upload the sample PDF unless email delivery is added later.

10. Minimal technical architecture

```
Browser / Next.js client
|-- File[] originals stay local in current tab
|-- EXIF + thumbnails + clustering + layouts
|-- Book preview
|-- pdf-lib generates final interior.pdf + cover.pdf
|
|--> Next.js server route: /api/story --> AI provider
|--> Next.js server route: /api/leads --> Supabase Postgres
|--> Next.js server route: /api/order-assets --> signed Supabase upload URLs
|--> /checkout --> /api/lulu/quote --> Lulu API
|           --> Stripe payment
|
Stripe webhook (server)
|--> create long-lived signed URLs for final PDFs
|--> POST Lulu print job
|--> save lulu_print_job_id/status in Supabase

Lulu webhook (server)
|--> update order status
|--> delete final PDFs after safe retention window
```

Backend definition

There is a backend, but no separate backend service. Use Next.js Route Handlers/serverless functions. The heavy album work remains on the client. Supabase is used for lightweight database records and temporary final-PDF storage only.

11. Stack and repository structure

Layer	Tool
Framework	Next.js App Router + TypeScript
Styling	Tailwind CSS; shadcn/ui optional
Client state	Zustand or simple reducer
Photo metadata	exifr
Client image processing	createImageBitmap + Canvas/OffscreenCanvas
PDF generation	pdf-lib
Database/storage	Supabase Postgres + private Storage bucket
Payments	Stripe
Story AI	Provider-agnostic server call; model in env var
Printing/fulfillment	Lulu Print API
Deployment	Vercel is the simplest default for Next.js


```
app/  
  page.tsx  
  checkout/page.tsx  
  order/[id]/page.tsx  
  api/story/route.ts  
  api/leads/route.ts  
  api/order-assets/route.ts  
  api/lulu/quote/route.ts  
  api/stripe/webhook/route.ts  
  api/lulu/webhook/route.ts  
components/  
  AlbumDropzone.tsx  
  ProcessingProgress.tsx  
  BookSizeSlider.tsx  
  ChapterEditor.tsx  
  BookPreview.tsx  
  EmailSampleModal.tsx  
lib/  
  photo/exif.ts  
  photo/process.ts  
  photo/cluster.ts  
  photo/quality.ts  
  book/layouts.ts  
  book/pagination.ts  
  book/interior-pdf.ts  
  book/cover-pdf.ts  
  lulu/client.ts  
  lulu/config.ts  
  pricing.ts  
  supabase/server.ts  
  stripe.ts  
store/  
  useOurTailTalesStore.ts  
types/  
  book.ts  
  photo.ts  
  order.ts
```

12. Supabase: keep it tiny

Do not store the customer's original album, regardless of whether it contains 50 photos or several thousand. Store business records and the two final print PDFs only.

Table	Minimum fields
leads	id, email, pet_name, photo_count, chapter_count, quoted_price, created_at, utm_*
orders	id, email, pet_name, chapter_count, story_pages, total_pages, book_price, shipping_price, stripe_payment_intent_id, status, lulu_print_job_id, interior_path, cover_path, created_at
order_shipping	order_id, name, phone, street1, street2, city, state, postcode, country, shipping_level

Storage

- Private bucket: ourtailtales-orders.
- Path: orders/{orderId}/interior.pdf and orders/{orderId}/cover.pdf.
- Client never receives Supabase service-role key.

- Generate signed upload URLs from a server route.
- At payment success, server creates signed download URLs long enough for Lulu to fetch/normalize the files.
- Delete PDFs after Lulu has safely accepted the order or after a short retention window. Early MVP fallback: daily cleanup job for assets older than 7 days that are not needed for an unresolved order.

No authentication in MVP

Use unguessable UUID order IDs and server-authorized operations. Do not add Supabase Auth unless customer demand proves that saved projects are necessary.

13. Lulu-ready PDF generation

Generate print files in the browser only after the customer is satisfied with the preview and clicks Order. This means ourTailTales uploads two finished PDFs instead of hundreds of source photos.

Interior PDF

- Trim: 8.5 x 8.5 in.
- Because ourTailTales uses full-bleed photography, render each PDF page at 8.75 x 8.75 in (0.125 in bleed on every edge).
- Use 300 PPI target for embedded raster assets; do not upscale tiny originals and pretend they are print quality.
- Keep important text inside a conservative safe area. Lulu recommends 0.5 in from trim; use ~0.625-0.75 in from document edge for text to remain safe across the expected page-count range.
- All pages must have identical dimensions; no crop/bleed marks.
- Embed fonts and flatten transparency where practical.

Cover PDF

- Single-page spread: back cover + spine + front cover.
- Do not hardcode the final cover canvas width from a guessed formula.
- Ask Lulu's cover-dimensions endpoint for the exact required dimensions using `pod_package_id` + page count, then generate the PDF to that exact size.
- Casewrap art must extend through Lulu's 0.75 in wrap region; keep critical text/images away from the hinge and wrap edges.
- Cover design for V1: full-bleed hero image, pet name, optional lifespan, ourTailTales mark on back. Keep spine text optional because the base book is thin.

Physical sample before launch

Order the exact V1 configuration from Lulu before accepting live customers. Verify crop tolerance, dark-photo reproduction, text size, hinge placement, spine behavior at ~44 pages, and matte vs gloss cover finish.

14. Lulu Print API integration

All Lulu credentials and calls live server-side. Never expose client key/secret in browser JavaScript.

V1 product configuration

- Trim: 8.5 x 8.5 square.
- Ink: full color.
- Quality: premium.

- Binding: hardcover casewrap.
- Paper: 80# coated white.
- Finish: choose matte for the initial sample; switch if the physical sample favors gloss.
- Store the verified current dotted-format pod_package_id in an environment variable. Do not hardcode the legacy 27-character SKU; Lulu ends legacy support in 2027.

```
LULU_ENV=sandbox
LULU_CLIENT_KEY=...
LULU_CLIENT_SECRET=...
LULU_POD_PACKAGE_ID=... // verified via Lulu Price Calculator, dotted format
LULU_CONTACT_EMAIL=orders@ourtailtales.com
```

Authentication

- OAuth/OpenID Connect client-credentials flow.
- POST Lulu token endpoint with grant_type=client_credentials and Basic auth made from client key/secret.
- Cache bearer access token server-side until near expiry; refresh when needed.
- Use sandbox account and https://api.sandbox.lulu.com/ throughout development.

Checkout quote flow

1. Checkout form collects customer name, phone, email, and U.S. shipping address.
2. POST server-side to Lulu print-job cost calculation/address validation endpoint using page count, POD package ID, quantity 1, address, and a candidate shipping level.
3. Use Lulu shipping options to offer only a small set: e.g., MAIL/GROUND and EXPEDITED when available.
4. Show customer shipping charge separately from ourTailTales book price.
5. Do not expose Lulu manufacturing cost to the customer.

Paid order flow

1. Customer pays ourTailTales via Stripe.
2. Stripe webhook verifies successful payment server-side.
3. Load order record and create signed download URLs for interior.pdf and cover.pdf.
4. Get Lulu bearer token.
5. POST /print-jobs/ with one line item, POD package ID, signed file URLs, quantity 1, title, shipping address, shipping level, contact email, and ourTailTales order ID as external_id.
6. Save Lulu print-job ID and initial status in Supabase.
7. Lulu separately charges ourTailTales's saved Lulu payment method via Auto Pay; the customer's Stripe payment does not directly pay Lulu.

Lulu webhooks

- Subscribe to PRINT_JOB_STATUS_CHANGED.
- Verify Lulu-HMAC-SHA256 using the raw request body before parsing JSON.
- Update Supabase order status: created/unpaid/production/shipped/delivered/rejected/etc.
- If Lulu rejects for PDF/SKU/file URL issues, flag order for manual review and do not silently recreate multiple paid print jobs.

Auto Pay is required for “straight away” fulfillment

Lulu can automatically charge a saved merchant card when API orders arrive. Enable Auto Pay in the Lulu developer account before production launch; otherwise print jobs may remain unpaid and will not move into production.

15. Lulu request shape (implementation guide)

Treat this as the shape Cursor should implement against the current Lulu OpenAPI schema; verify exact field names in the live sandbox docs while coding.

```
POST https://api.sandbox.lulu.com/print-jobs/
Authorization: Bearer <token>
Content-Type: application/json

{
  "contact_email": "orders@ourtailtales.com",
  "external_id": "<ourtailtales-order-uuid>",
  "shipping_level": "GROUND",
  "shipping_address": {
    "name": "Customer Name",
    "phone_number": "+1...",
    "street1": "...",
    "street2": "",
    "city": "...",
    "state_code": "TX",
    "postcode": "...",
    "country_code": "US"
  },
  "line_items": [{
    "external_id": "<ourtailtales-order-uuid>-book",
    "title": "<Pet Name> - ourTailTales",
    "quantity": 1,
    "printable_normalization": {
      "pod_package_id": "<LULU_POD_PACKAGE_ID>",
      "interior": { "source_url": "<signed-interior-url>" },
      "cover": { "source_url": "<signed-cover-url>" }
    }
  }]
}
```

// IMPORTANT: exact nesting/field names must be confirmed against current Lulu sandbox OpenAPI.

Why the note above matters

Lulu's API has evolved, including the 2026 POD package ID migration. Cursor should use Lulu's current OpenAPI schema as the source of truth instead of copying old GitHub examples verbatim.

16. Stripe and checkout

Use Stripe only for ourTailTales's customer payment. Lulu remains the downstream manufacturer that bills ourTailTales separately.

- Book price: derived only from chapter count.
- Shipping: quoted from Lulu and passed through to customer (optionally round up slightly later).
- Tax: for a public launch, use Stripe Tax or obtain tax advice. Do not invent a hardcoded sales-tax rule in application code.
- Create PaymentIntent server-side after shipping quote is locked.
- Store order as pending_payment before payment.
- Only Stripe webhook may transition order to paid and submit Lulu print job.
- Make Stripe webhook idempotent. If the event is delivered twice, never create two Lulu print jobs.

```
OURTAILTALES / MVP BUILD SPEC
if (order.lulu_print_job_id) return; // idempotency guard
if (payment.status !== 'succeeded') return;

const urls = await createSignedPrintAssetUrls(order);
const printJob = await lulu.createPrintJob(order, urls);
await markOrderSubmitted(order.id, printJob.id);
```

17. Failure and edge-case behavior

Failure	MVP behavior
No EXIF dates	Use lastModified/file order; show “Some dates are approximate.”
No GPS	Create chapters from time only; omit location language
Few photos	Lower recommended chapter count; never force 20-30 photos/chapter
Many duplicates	Keep best representative and allow user to restore hidden duplicates
Low-res photo selected	Warn on that page; allow customer to keep it
AI fails	Retry per chapter; user can manually type title/blurb
Tab close/refresh	Standard beforeunload warning; progress is lost
PDF generation fails	Keep editor state in current tab and let user retry
Lulu quote/address warning	Show suggested corrected address before payment
Lulu print job rejected	Mark order needs_review; do not duplicate charge/order automatically

18. Minimal analytics and privacy

Track funnel conversion without uploading personal photos.

- landing_view
- album_selected (count only)
- processing_complete
- book_size_confirmed (chapter count / price)
- story_generated
- sample_email_submitted
- checkout_started
- payment_succeeded
- lulu_submitted

Privacy copy should clearly state:

- Original album photos remain in the browser during creation.
- Only small selected thumbnails are transmitted when AI story generation is requested.
- When the user orders, ourTailTales creates and temporarily stores the final print PDFs so Lulu can manufacture the book.
- Temporary print files are deleted after fulfillment/retention period.

19. Cursor implementation order

1. Create Next.js app shell, ourTailTales visual system, landing hero, dropzone, and state machine.
2. Implement album ingestion, EXIF extraction, thumbnails, progress, beforeunload warning.
3. Implement chronology + duplicate/quality scoring + deterministic chapter clustering.
4. Implement chapter slider/pricing and selected-photo logic.
5. Implement fixed page templates and live browser book preview.
6. Implement server AI route and structured chapter title/blurb generation.
7. Implement lightweight editor controls.
8. Implement local 5-page watermarked PDF + Supabase lead capture.
9. Implement full Lulu interior PDF export.
10. Implement Lulu cover-dimensions call + cover PDF export.
11. Implement Supabase private storage + signed upload/download URLs.
12. Implement checkout address/phone form + Lulu quote/shipping options.
13. Implement Stripe PaymentIntent + webhook idempotency.
14. Implement Lulu sandbox auth + print-job submission + webhook status updates.
15. Run end-to-end sandbox order.
16. Order physical Lulu test book and correct layout/cover issues before production credentials are enabled.

20. MVP acceptance criteria

- Albums of widely varying sizes (for example 50 to 2,000+ photos) can be selected without uploading originals to ourTailTales, while processing remains incremental and memory-bounded.
- Processing stays responsive and visibly progresses instead of freezing the page.
- Detected timeline is chronologically reasonable and chapter count changes cleanly with slider.
- Default ~4-chapter book produces 40 story pages + 4 fixed pages.
- AI writes only chapter-level copy and does not claim unsupported memories.
- User can preview every page and swap/reorder selected photos.
- 5-page watermarked sample downloads after email capture.
- Final interior/cover PDFs pass Lulu sandbox validation for the configured SKU.
- Checkout obtains a valid Lulu shipping option/address validation before payment.
- Successful Stripe webhook creates exactly one Lulu print job.
- Lulu status webhook updates order record.
- A physical sample proves acceptable print quality before public launch.

Do not expand scope before these pass
No accounts, no saved drafts, no additional sizes, no mobile-native app, no page-by-page AI captions, and no custom free-form editor until the purchase funnel works end to end.

21. Recommended launch defaults

Decision	Default
----------	---------

Recommended chapter count	4 chapters when album supports it
Recommended customer price	\$69.99 + shipping/tax
Story pages	40
Actual Lulu interior pages	44
Selected photos	~80-110 total
AI thumbnails	3-5 representative thumbnails per chapter
Chapter blurb	60-110 words
Shipping geography	United States only for initial launch
Quantity	1 copy; quantity selector can come later
Cover	Casewrap, one hero photo, pet name, lifespan
User accounts	None
File retention	Short-lived final PDFs only

22. Environment variables

```

NEXT_PUBLIC_SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_STORAGE_BUCKET=ourtailtales-orders

```

```

AI_PROVIDER=
AI_API_KEY=
AI_MODEL=

```

```

STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=

```

```

LULU_ENV=sandbox
LULU_CLIENT_KEY=
LULU_CLIENT_SECRET=
LULU_POD_PACKAGE_ID=
LULU_CONTACT_EMAIL=
LULU_WEBHOOK_SECRET=

```

```

NEXT_PUBLIC_SITE_URL=http://localhost:3000
# production: https://ourtailtales.com

```

23. Lulu facts verified for this spec

- Lulu lists Square 8.5 x 8.5 in hardcover photo books with Premium Color and 80# paper, starting at \$14.76 before shipping/tax/fees.
- The square hardcover casewrap configuration supports a minimum of 24 pages and up to 800 pages.
- Lulu requires a multipage interior PDF and a single-page cover PDF accessible through download URLs.
- Full-bleed interiors use 0.125 in bleed; a square 8.5 x 8.5 page is therefore 8.75 x 8.75 in when full bleed is included.
- Hardcover casewrap cover files require additional wrap area; Lulu provides/returns dimensions based on page count and product configuration.
- The Print API uses OAuth/OpenID Connect client credentials and has a sandbox environment.
- Lulu supports print-job cost/address validation, shipping options, print-job creation/status, and webhooks.

- Lulu's 2026 dotted POD package ID format is live; legacy non-dotted support is scheduled to end February 1, 2027.
- Lulu API use has no subscription/startup fee; order charges include printing, shipping, fulfillment fee, and applicable tax.
- Lulu Auto Pay can automatically charge the merchant payment method for API orders.

Primary references

Lulu Photo Books: <https://www.lulu.com/create/photo-books>

Lulu Developer API Documentation: <https://lulu.apidocumentation.com/>

Lulu Developer Knowledge Base: <https://help.api.lulu.com/>

Lulu PDF Creation Settings: <https://help.api.lulu.com/en/support/solutions/articles/64000254609-pdf-creation-settings>

Lulu Casewrap Cover Guide: <https://help.lulu.com/en/support/solutions/articles/64000308572-creating-your-hardcover-casewrap-cover>

Lulu Print API file requirements: <https://help.api.lulu.com/en/support/solutions/articles/64000254607-what-files-are-required-for-lulu-print-api-production->

Lulu Auto Pay: <https://help.api.lulu.com/en/support/solutions/articles/64000293838-how-do-i-set-up-automatic-payment-fulfillment-for-my-api-orders->

Lulu Sandbox: <https://help.api.lulu.com/en/support/solutions/articles/64000306383-do-you-have-a-sandbox-environment->

Implementation note

Cursor should verify the live Lulu OpenAPI schema while coding.

This document defines ourTailTales's product architecture and the intended Lulu flow. Where API request field names evolve, the live Lulu sandbox OpenAPI specification overrides example payload syntax in this brief.